

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студент потока характеризуется следующими данными:

- ФИО (до 50 символов);
- номер группы;
- набор из пяти оценок за последнюю сессию (без указания предметов);
- размер стипендии.
- дополнительная информация (выбирается индивидуально каждым студентом,

например, телефон, эл. почта, год рождения и т.п.)

Необходимо:

1) разработать (и программно реализовать) динамические структуры данных и

алгоритмы их обработки, позволяющие поддерживать выполнение следующих функций:

- консольный ввод/вывод данных о всех студентах потока;
- файловый ввод/вывод данных о потоке;
- редактирование данных о студентах и группах потока, включающее операции

добавления/удаления групп и студентов;

2) разработать и программно реализовать алгоритмы обработки базы данных, предусмотренные персональным заданием

Индивидуальное задание:

Вывести перечень номеров групп, упорядоченный по значению показателя

«число двоечников» / «списочная численность группы»

Требования к программе:

- 1) Программа должна поддерживать систему меню, пункты которых соответствуют выполнению функций, предусмотренных общей частью задания.
- 2) Предлагаемые структуры данных должны учитывать изначальную неопределенность возможного количества групп и студентов в группах, а также обеспечивать максимальную скорость процессов обработки данных, предусмотренных заданием.
- 3) Тексты программ должны содержать комментарии, объясняющие назначение основных функций, типов и объектов данных, функциональных блоков и т.п.
- 4) Представляемые тексты программ должны обеспечивать возможность их компиляции и построения в среде MS Visual Studio

Содержание пояснительной записки:

- 1) Оглавление
- 2) Содержание работы
- 3) Вывод
- 4) Список использованных источников
- 5) Приложение

АННОТАЦИЯ

Работа посвящена разработке базе данных «Студенческий поток».

Программа создается на языке программирования C++ без использования сторонних библиотек.

В результате работы была успешно получена программа, позволяющая создать базу данных с информацией о студентах потока, осуществлять ввод/вывод данных через консоль и/или текстовый файл, а также свободно удалять и редактировать информацию об отдельном студенте или целой группе.

ВВЕДЕНИЕ

Описание задания

Общая часть задания

Студент потока характеризуется следующими данными:

- ФИО (до 50 символов);
- номер группы;
- набор из пяти оценок за последнюю сессию (без указания предметов);
- размер стипендии.
- дополнительная информация (выбирается индивидуально каждым студентом, например, телефон, эл. почта, год рождения и т.п.)

Необходимо:

1) разработать (и программно реализовать) динамические структуры данных и

алгоритмы их обработки, позволяющие поддерживать выполнение следующих функций:

- консольный ввод/вывод данных о всех студентах потока;
- файловый ввод/вывод данных о потоке;
- редактирование данных о студентах и группах потока, включающее операции

добавления/удаления групп и студентов;

2) разработать и программно реализовать алгоритмы обработки базы данных, предусмотренные персональным заданием

Индивидуальное задание:

Вывести перечень номеров групп, упорядоченный по значению показателя
«число двоечников» / «списочная численность группы»

Требования к программе:

- 1) Программа должна поддерживать систему меню, пункты которых соответствуют выполнению функций, предусмотренных общей частью задания.
- 2) Предлагаемые структуры данных должны учитывать изначальную неопределенность возможного количества групп и студентов в группах, а также обеспечивать максимальную скорость процессов обработки данных, предусмотренных заданием.
- 3) Тексты программ должны содержать комментарии, объясняющие назначение основных функций, типов и объектов данных, функциональных блоков и т.п.
- 4) Представляемые тексты программ должны обеспечивать возможность их компиляции и построения в среде MS Visual Studio

СОДЕРЖАНИЕ РАБОТЫ

Общая структура программы

Программа состоит из функции `main()`, внутри которой происходит только объявление глобальных переменных, а также создание меню, через которое происходит вызов остальных функций. Все остальные функции программы (ввод/вывод матриц, изменение элементов матриц, поиск их определителей и т.д.) выведены в отдельные функции.

Функция `main()`:

Алгоритм создания функции `main()` (код программы представлен в приложении):

- 1) Объявляем все необходимые переменные;
- 2) Создаём «нулевой» элемент списка групп;
- 3) Используя цикл `while` создаем меню;
- 4) С помощью условного оператора `if` вызываем функции программы в зависимости от выбора пользователя;
- 5) Прописываем выход из цикла по желанию пользователя;
- 6) Очищаем память;
- 7) Завершаем работу программы;

Структуры узлов студентов и групп:

Студента:

```
struct Node_student //
{
    string fio; //фio
    int group; // ном. группы
    int marks[5]; //массивоценок
    int stip; //размерстипендии
    int age; //возраст
    Node_student *next_st; //указатели на след. и пред. студента группы
    Node_student *prev_st;
```

```
};
```

Группы:

```
struct Node_group //структура для группы
```

```
{
```

```
    int gr_num; //номер группы
```

```
    Node_group *next_group; //указатели на след., пред. группы и первого студента группы
```

```
    Node_group *prev_group;
```

```
    Node_student *stud;
```

```
};
```

Функции ввода:

1) Ввод с консоли:

Ввод с консоли обеспечивается функцией `add_stud_con()`.

Принцип работы функции:

А) Создаем новый узел списка студентов;

Б) Запрашиваем у пользователя информацию о студенте;

В) Ищем узел списка групп с указанным номером;

Г) Если такой группы ещё нет – создаем с помощью функции `add_group()`

Д) Если студент первый в группе – указатель в узле группы будет указывать на него, иначе – указатель на него будет в узле предыдущего студента;

2) Ввод из файла:

Ввод из файла обеспечивается функцией `create_node()`:

Когда пользователь выбирает ввод из файла, ещё внутри функции `main()` считывается первая строка из файла, в котором содержится количество студентов и запускается цикл `for`, в котором соответствующее количество раз считывается набор данных о студенте. Эти данные передаются в функцию `create_node()`, которая полностью аналогична функции `add_stud_con()` за

исключением того, что данные о студенте уже известны и передаются в функцию.

3) Создание узла группы:

Создание узла группы происходит с использованием функции `add_group()`. Принцип её работы очевиден – создается новый экземпляр структуры с указанными значениями атрибутов и, с помощью указателей, вставляется в конец списка групп.

Вывод информации о студентах:

Для вывода информации о студентах не было написано отдельных функций, так как это действие выполняется в несколько строчек кода и создание для этого отдельных функций является избыточным.

Когда пользователь выбирает вывод информации о студентах, список групп проходится от первой до последней, по мере прохождения списка проходятся списки студентов каждой группы и информация о каждом студенте выводится в консоль/файл.

Редактирование информации о студенте:

Возможность редактировать информацию о студенте реализована с помощью функции `red_info()`. Ещё внутри функции `main()`, перед вызовом функции `ref_info()`, у пользователя запрашивается номер группы и ФИО студента, информацию о котором нужно редактировать. После этого, с помощью цикла `while`, осуществляется поиск этого студента. Если он не найден – выводим соответствующее сообщение в консоль. Иначе – передаем указатель на студента в функции `red_info()`.

Принцип работы функции `red_info()`:

Запрашиваем, с помощью небольшого меню, у пользователя какую информацию он хочет изменить.

Если это возраст/оценки/размер стипендии, то новая информация сразу вносится в узел.

Если требуется изменить номер группы студента, то создаются временные переменные для остальной информации о нём, после чего, с помощью

нижеописанной функции, узел удаляется и, через функцию `create_node()`, создается новый, уже в новой группе

Если измененный узел был единственным в своей группе, то пустая группа удаляется.

Редактирование информации о группе:

Так как единственная информация о группе – это её номер, её изменение также происходит не внутри отдельной функции, а прямо внутри `main()`. У пользователя запрашивается старый номер группы, ищется группа с таким номером и, если она найдена, вводится новый номер.

Функции для удаления информации:

Удаление всех студентов группы обеспечивается функцией `del_stud_all()`.

Принцип работы функции элементарен – в нее передается указатель на первый узел списка. Если он не последний, то функция вызывается рекурсивно для следующего узла. Иначе узел удаляется

Удаление информация об одном студенте происходит с помощью функции `del_stud()`. Она принимает указатель на первый элемент списка и ФИО студента. Если такой студент найден, информация о нем удаляется, указатели следующего и предыдущего(если они есть) изменяются соответствующим образом

Удаление информации о группе осуществляется функцией `del_gr()`. Функция принимает указатель на удаляемую группу. Внутри функции ряд удаляется, указатели следующего и предыдущего узлов(если они есть) изменяются

Функция для выполнения индивидуального задания:

Для выполнения индивидуального задания создана функция `task()`. Она принимает в себя указатель на первый элемент списка групп.

Принцип работы функции:

- 1) Создается 3 массива, длины которых равны количеству групп.
- 2) Первый массив заполняется номерами групп.

- 3) Второй, с помощью цикла `while`, заполняется численностями групп таким образом, чтобы индекс номера группы в одном массиве и индекс её численности в другом совпадали.
- 4) Третий массив, с помощью цикла `while` и условного оператора `if` аналогичным образом заполняется количеством двоечников в каждой из групп
- 5) Третий массив сортируется пузырьковым методом. При этом, при изменении элементов этого массива, соответствующим образом меняются элементы других массивов так, чтобы не нарушалось соответствие индексов
- 6) После происходит сортировка второго массива по возрастанию, однако два элемента меняются местами только в том случае, если соответствующие им значения в третьем массиве равны
- 7) Отсортированная информация выводится в консоль.

Код:

```
#include <iostream>
#include <cstdlib>
#include <string>
#include <windows.h>
#include <fstream>
```

```
using namespace std;
```

```
struct Node_student //структура для студента
{
    string fio; //фio
    int group; // ном. группы
    int marks[5]; //массив оценок
    int stip; //размер стипендии
    int year_of_birth; //год рождения
    Node_student *next_st; //указатели на след. и пред. студента группы
    Node_student *prev_st;
};
```

```
struct Node_group //структура для группы
{
    int gr_num; //номер группы
    Node_group *next_group; //указатели на след., пред. группы и первого студента
    группы
    Node_group *prev_group;
    Node_student *stud;
};
```

```
void add_group(Node_group* ptr, int num) //создание группы
```

```

{
    Node_group *ngroup = nullptr;
    ngroup = new Node_group;
    ngroup -> prev_group = ptr;
    ngroup -> next_group = nullptr;
    ngroup -> gr_num = num;
    ngroup -> stud = nullptr;
    ptr -> next_group = ngroup;
}

```

```

void add_stud_con(Node_group *ptr_gr) //создание карточки студента через консоль
{
    Node_student *nstud = nullptr; //создаем новый узел
    nstud = new Node_student;
    nstud -> next_st = nullptr;
    wcout << L"Введите номер группы: "; //запрашиваем всю нужную информацию
    cin >> nstud -> group;
    wcout << L"Введите ФИО студента: ";
    string s;
    getline(cin, s);
    getline(cin, nstud -> fio);
    wcout << L"Введите размер стипендии: ";
    cin >> nstud -> stip;
    wcout << L"Введите год рождения: ";
    cin >> nstud -> year_of_birth;
    wcout << L"Последовательно(через Enter) введите оценки студента за сессию:" <<
endl;
    for (int j = 0; j < 5; j++)
    {
        cin >> nstud -> marks[j];
    }
    while (ptr_gr -> gr_num != nstud -> group) //находим группу с нужным номером
    {
        if (ptr_gr -> next_group == nullptr) {break;}
    }
}

```

```

        ptr_gr = ptr_gr -> next_group;
    }
    if (ptr_gr -> gr_num != nstud -> group) //если такой группы нет - создаем
    {
        add_group(ptr_gr, nstud -> group);
        ptr_gr = ptr_gr -> next_group;
    }
    if (ptr_gr -> stud == nullptr) //если в группе нет студентов, введенный будет первым
    {
        ptr_gr -> stud = nstud;
        nstud -> prev_st = nullptr;
    }
    else //иначе пробегаем список студентов группы и вставляем нового в конец
    {
        Node_student *ptr_st1 = ptr_gr -> stud;
        while (ptr_st1 -> next_st != nullptr) {ptr_st1 = ptr_st1 -> next_st;}
        ptr_st1 -> next_st = nstud;
        nstud -> prev_st = ptr_st1;
    }
}

```

```

void create_node(Node_group *ptr_gr, string fio, int stip, int year_of_birth, int marks[])
//создает карточку студента с уже известными данными(для файлового ввода)
{
    Node_student *nstud = nullptr; //функция полностью аналогична предыдущей
    nstud = new Node_student;
    nstud -> next_st = nullptr;
    nstud -> fio = fio;
    nstud -> stip = stip;
    nstud -> year_of_birth = year_of_birth;
    nstud -> marks[0] = marks[0];
    nstud -> marks[1] = marks[1];
    nstud -> marks[2] = marks[2];
    nstud -> marks[3] = marks[3];
}

```

```

nstud -> marks[4] = marks[4];
if (ptr_gr -> stud == nullptr)
    {
        ptr_gr -> stud = nstud;
        nstud -> prev_st = nullptr;
    }
else
    {
        Node_student *ptr_st1 = ptr_gr -> stud;
        while (ptr_st1 -> next_st != nullptr) {ptr_st1 = ptr_st1 -> next_st;}
        ptr_st1 -> next_st = nstud;
        nstud -> prev_st = ptr_st1;
    }
}

```

```

void del_stud_all(Node_student *ptr) //рекурсивное удаление всех студентов группы
{
    if (ptr -> next_st != nullptr)
    {
        del_stud_all(ptr -> next_st);
    }
    delete ptr;
}

```

```

void del_gr(Node_group *ptr) //удаление группы
{
    Node_group *temp_next = ptr -> next_group;
    Node_group *temp_prev = ptr -> prev_group;
    if (ptr -> stud != nullptr) //проверка на наличие студентов в группе
    {
        del_stud_all(ptr -> stud); //удаление всех студентов группы
    }
}

```

```

    }
    if (ptr -> next_group != nullptr)
    {
        ptr -> next_group -> prev_group = temp_prev;
    }
    if (ptr -> prev_group != nullptr)
    {
        ptr -> prev_group -> next_group = temp_next;
    }
    delete ptr;
}

void del_stud(Node_group *ptr_gr, string fio) //удаление одного студента
{
    Node_student *ptr = ptr_gr -> stud;
    if (ptr == nullptr)
    {
        wcout << L"Такого студента не найдено!" << endl;
        return;
    }
    while (ptr -> fio != fio) //циклом ищем студента
    {
        ptr = ptr -> next_st;
        if (ptr == nullptr)
        {
            wcout << L"Такого студента не найдено!" << endl; //если нет -
сообщаем об этом
            return;
        }
    }
    Node_student *temp_next = ptr -> next_st; //если есть - удаляем
    Node_student *temp_prev = ptr -> prev_st;
    if (temp_prev != nullptr)
    {

```

```

        temp_prev -> next_st = temp_next;
    }
    if (temp_next != nullptr)
    {
        temp_next -> prev_st = temp_prev;
    }
    if (ptr_gr -> stud == ptr)
    {
        ptr_gr -> stud = ptr -> next_st;
    }
    delete ptr;
}

```

```

void red_info(Node_group* ptr_gr)
{
    Node_group *ptr_fgr = ptr_gr;
    int group;
    string fio, s;
    system("cls");
    wcout << L"Введите номер группы студента, информацию о которой вы хотите
редактировать: ";
    cin >> group;
    while (ptr_gr -> gr_num != group) //Ищем такую группу
    {
        ptr_gr = ptr_gr -> next_group;
        if (ptr_gr == nullptr)
        {
            wcout << L"Такой группы не найдено!" << endl;
            return;
        }
    }
    Node_student *ptr_st = ptr_gr -> stud;
    wcout << L"Введите ФИО студента: ";
    getline(cin, s);
}

```



```

getline(cin, fio);
if (ptr_st == nullptr)
{
    wcout << L"Такого студента не найдено!" << endl;
    return;
}
while (ptr_st -> fio != fio) //Ищем такого студента
{
    ptr_st = ptr_st -> next_st;
    if (ptr_st == nullptr)
    {
        wcout << L"Такого студента не найдено!" << endl;
        return;
    }
}
int choice; //создаем подменю
while (true)
{
    system("cls");
    int new_group; //переменная для новой группы
    wcout << L"Выберите действие:" << endl << L"1 - изменить размер
стипендии" << endl << L"2 - изменить год рождения" << endl //менюшка
    << L"3 - изменить оценки" << endl << L"4 - изменить номер группы" << endl;
    cin >> choice;
    while ((choice > 4)|| (choice < 1))
    {
        wcout << L"Введите корректное число: " << endl;
        cin >> choice;
    }
    wcout << L"Введите новые данные: ";
    if (choice == 1) //в соответствии с выбранным вариантом заменяем данные
    {
        cin >> ptr_st -> stip;
    }
    if (choice == 2)

```

```

    {
        cin >> ptr_st -> year_of_birth;
    }
    if (choice == 3)
    {
        cout << endl;
        for (int j = 0; j < 5; j++)
        {
            cin >> ptr_st -> marks[j];
        }
    }
    if (choice == 4) //при замене группы
    {
        cin >> new_group; //принимаем новый номер
        int temp_stip = ptr_st -> stip; //создаем временные переменные для
данных о студенте
        int temp_year_of_birth = ptr_st -> year_of_birth;
        int temp_marks[5];
        for (int j = 0; j < 5; j++)
        {
            temp_marks[j] = ptr_st -> marks[j];
        }
        string temp_fio = ptr_st -> fio;
        del_stud(ptr_gr, temp_fio); //удаляем старый узел
        if (ptr_gr -> stud == nullptr)
        {
            del_gr(ptr_gr);
        }
        Node_student *nstud = nullptr; //создаем новый узел(следующий код
полностью аналогичен функции add_stud_con())
        nstud = new Node_student;
        nstud -> next_st = nullptr;
        nstud -> group = new_group;
        nstud -> fio = temp_fio;
        nstud -> stip = temp_stip;

```

```

nstud -> year_of_birth = temp_year_of_birth;
for (int j = 0; j < 5; j++)
{
    nstud -> marks[j] = temp_marks[j];
}
ptr_gr = ptr_fgr;
while (ptr_gr -> gr_num != nstud -> group)
{
    if (ptr_gr -> next_group == nullptr) {break;}
    ptr_gr = ptr_gr -> next_group;
}
if (ptr_gr -> gr_num != nstud -> group) //если такой группы нет -
создаем
{
    add_group(ptr_gr, nstud -> group);
    ptr_gr = ptr_gr -> next_group;
}
if (ptr_gr -> stud == nullptr) //если в группе нет студентов, введенный
будет первым
{
    ptr_gr -> stud = nstud;
    nstud -> prev_st = nullptr;
}
else //иначе пробегаем список студентов группы и вставляем нового в
конец
{
    Node_student *ptr_st1 = ptr_gr -> stud;
    while (ptr_st1 -> next_st != nullptr) {ptr_st1 = ptr_st1 -> next_st;}
    ptr_st1 -> next_st = nstud;
    nstud -> prev_st = ptr_st1;
}
}
wcout << L"Готово!" << endl;
break;
}

```

```
}
```

```
double percent_calculation(Node_group *group_ptr) //вычисление процента двоечников и  
троечников в группе
```

```
{
```

```
    Node_student *student_ptr = group_ptr -> stud;
```

```
    double count_all = 0;
```

```
    double count = 0;
```

```
    if (student_ptr == nullptr) //если в группе нет студентов
```

```
    {
```

```
        return -1; //возвращаем -1(чтобы эта группа не выводилась в конце в списке
```

```
групп)
```

```
    }
```

```
    while (student_ptr != nullptr)
```

```
    {
```

```
        count_all++; //считаем общее кол-во студентов
```

```
        for (int j = 0; j < 5; j++)
```

```
        {
```

```
            if (student_ptr -> marks[j] < 4) //и кол-во двоечников/троечников
```

```
            {
```

```
                count++;
```

```
                break;
```

```
            }
```

```
        }
```

```
        student_ptr = student_ptr -> next_st;
```

```
    }
```

```
    return count/count_all;
```

```
}
```

```
void ind_zad(Node_group *group_ptr1)
```

```
{
```

```
    int group_amount = 0; //переменная для количества групп
```

```
    Node_group *group_ptr = group_ptr1; //рабочие указатели
```

```

Node_student *student_ptr = nullptr;
int a; //вспомогательные переменные, нужны при сортировке
double b;
while (group_ptr != nullptr) //считаем кол-во групп
{
    group_amount++;
    group_ptr = group_ptr -> next_group;
}
int group_numbers[group_amount]; //создаем массивы для номеров групп и
процентов двоечников и троечников в этих группах
double percents[group_amount];
group_ptr = group_ptr1;
for (int i = 0; i < group_amount; i++)
{
    group_numbers[i] = group_ptr -> gr_num; //заполняем первый массив
номераами групп
    group_ptr = group_ptr -> next_group;
    percents[i] = 0; //второй - нулями
}
group_ptr = group_ptr1;
for (int i = 0; i < group_amount; i++)
{
    percents[i] = percent_calculation(group_ptr)*100;
    group_ptr = group_ptr -> next_group;
}
for (int i = 0; i < group_amount; i++) //сортируем по номерам групп
{
    for (int j = 0; j < group_amount - 1; j++)
    {
        if (group_numbers[j] > group_numbers[j+1])
        {
            a = group_numbers[j];
            group_numbers[j] = group_numbers[j+1];
            group_numbers[j+1] = a;
            b = percents[j];

```

```

        percents[j] = percents[j+1];
        percents[j+1] = b;
    }
}

int limit_percent = 0;
wcout << L"Введите предел доли двоечников/троечников в группе(в процентах): ";
cin >> limit_percent;
wcout << L"Группы, в которых двоечников/троечников более " << limit_percent <<
"%:" << endl;
for (int i = 0; i < group_amount; i++)
{
    if ((percents[i] > limit_percent)&&(percents[i] >= 0))
    {
        cout << group_numbers[i] << endl;
    }
}
}

```

```

int main()
{
    fstream file_out; //переменные для файлов
    fstream file_in;
    setlocale(LC_ALL, "RUSSIAN");
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    int choice = 0; //переменная для меню
    int amount; //вспом. переменная
    int gr_to_del; //группа на удаление
    int gr_to_change; //группа на редактирование
    int year_of_birth_from_file; //переменные для данных из файла
    int group_from_file;
    int stip_from_file;
    string fio_from_file;
}

```

```

int marks_from_file[5];
bool flag = false; //вспом. переменная
string s, fio_to_del; //вспом. переменная и фамилия студента на удаление
Node_group *first_group = nullptr; //первый элемент списка групп
first_group = new Node_group;
first_group -> stud = nullptr;
first_group -> next_group = nullptr;
first_group -> prev_group = nullptr;
first_group -> gr_num = 0;
Node_group *ptr_gr = first_group; //рабочие указатели
Node_student *ptr_st = nullptr;
while (true) //цикл с менюшкой
{
    ptr_gr = first_group;
    system("cls");
    wcout << L"Выберите действие:" << endl << L"1 - вывести информацию о
студентах в консоль" << endl << L"2 - вывести информацию о студентах в файл" <<
endl
    << L"3 - ввести информацию о студентах с консоли" << endl << L"4 - ввести
информацию о студентах из файла" << endl << L"5 - удалить информацию о группе"
<<
    endl << L"6 - удалить информацию о студенте" << endl << L"7 -
редактировать информацию о студенте" << endl << L"8 - изменить номер группы" <<
endl
    << L"9 - вывести номера групп, в которых доля двоечников превышает
определенный процент" << endl
    << L"10 - завершить работу программы" << endl;
    cin >> choice;
    while ((choice > 10)|| (choice < 1))
    {
        wcout << L"Введите корректное число: " << endl;
        cin >> choice;
    }
    system("cls");
    if (choice == 1) //Вывод в консоль

```

```

{
    while (ptr_gr != nullptr)
    {
        if (ptr_gr -> stud != nullptr)
        {
            wcout << L"Группа №" << ptr_gr -> gr_num << endl;
            ptr_st = ptr_gr -> stud;
            while (ptr_st != nullptr)
            {
                cout << ptr_st -> fio << endl;
                wcout << L"  Размер стипендии: " << ptr_st ->
stip << endl;

                wcout << L"  Год рождения: ";
                cout << ptr_st -> year_of_birth << endl;
                wcout << L"  Оценки за последнюю сессию: ";
                for (int i = 0; i < 5; i++)
                {
                    cout << ptr_st -> marks[i] << " ";
                }
                cout << "" << endl;
                ptr_st = ptr_st -> next_st;
            }
        }
        ptr_gr = ptr_gr -> next_group;
    }
}

if (choice == 2) //вывод в файл
{
    file_out.open("output.txt", fstream::out);
    while (ptr_gr != nullptr)
    {
        if (ptr_gr -> stud != nullptr)
        {
            file_out << "Группа №" << ptr_gr -> gr_num << "\n";
            ptr_st = ptr_gr -> stud;

```



```

        while (ptr_st != nullptr)
        {
            s = " Размер стипендии: ";
            file_out << ptr_st -> fio << "\n";
            file_out << s << ptr_st -> stip << "\n";
            file_out << " Год рождения: " << ptr_st ->
year_of_birth << "\n";

            file_out << " Оценки за последнюю сессию: ";
            for (int i = 0; i < 5; i++)
            {
                file_out << ptr_st -> marks[i] << " ";
            }
            file_out << "" << "\n";
            ptr_st = ptr_st -> next_st;
        }
    }
    ptr_gr = ptr_gr -> next_group;
}
file_out.close();
wcout << L"Готово! Данные записаны в файл output.txt" << endl;
}
if (choice == 3) //ввод с консоли
{
    wcout << L"Введите количество студентов: ";
    cin >> amount; //запрашиваем кол-во студентов
    for (int i=0; i < amount; i++) //циклично вызываем функцию
    {
        wcout << L"Студент №" << i + 1 << ":" << endl;
        add_stud_con(ptr_gr);
        ptr_gr = first_group;
    }
}
if (choice == 4) //ввод из файла
{
    file_in.open("input.txt", fstream::in);

```

```

file_in >> amount; //первая строка - кол-во студентов
for (int j = 0; j < amount; j++) //циклично считываем данные
{
    file_in >> group_from_file;
    getline(file_in, s);
    getline(file_in, fio_from_file);
    file_in >> stip_from_file;
    file_in >> year_of_birth_from_file;
    for (int k = 0; k < 5; k++)
    {
        file_in >> marks_from_file[k];
    }
    while (ptr_gr -> gr_num != group_from_file)
    {
        if (ptr_gr -> next_group == nullptr) {break;}
        ptr_gr = ptr_gr -> next_group;
    }
    if (ptr_gr -> gr_num != group_from_file)
    {
        add_group(ptr_gr, group_from_file);
        ptr_gr = ptr_gr -> next_group;
    }
    create_node(ptr_gr, fio_from_file, stip_from_file,
year_of_birth_from_file, marks_from_file);
    ptr_gr = first_group;
}
file_in.close();
wcout << L"Готово!" << endl;
}
if (choice == 5) //удаление группы
{
    wcout << L"Введите номер группы, данные о которой вы хотите
удалить: ";

    cin >> gr_to_del; //запрашиваем номер
    while (ptr_gr -> gr_num != gr_to_del) //ищем такую группу

```

```

    {
        ptr_gr = ptr_gr -> next_group;
        if (ptr_gr == nullptr) //если не найдено
        {
            wcout << L"Такой группы не найдено!" << endl;
            break;
        }
    }
    if (ptr_gr != nullptr) //если найдено - вызываем соотв. функцию
    {
        del_gr(ptr_gr);
    }
    wcout << L"Готово!" << endl;
}
if (choice == 6) //удаление студента
{
    wcout << L"Введите номер группы студента, информацию о котором
вы хотите удалить: ";
    cin >> gr_to_del; //Принимаем номер группы
    while (ptr_gr -> gr_num != gr_to_del) //Ищем такую группу
    {
        ptr_gr = ptr_gr -> next_group;
        if (ptr_gr == nullptr)
        {
            wcout << L"Такой группы не найдено!" << endl;
            break;
        }
    }
    if (ptr_gr != nullptr) //если найдена
    {
        wcout << L"Введите ФИО студента: ";
        if (!flag)
        {
            getline(cin, s);
            getline(cin, fio_to_del);

```

```

        flag = true;
    }
    else
    {
        getline(cin, fio_to_del);
    }
    del_stud(ptr_gr, fio_to_del);
}
wcout << L"Готово!" << endl;
}
if (choice == 7) //ред. информации
{
    red_info(first_group);
}
if (choice == 8)
{
    wcout << L"Введите номер группы, номер которой хотите изменить: ";

    cin >> gr_to_change; //запрашиваем номер
    while (ptr_gr -> gr_num != gr_to_change) //ищем такую группу
    {
        ptr_gr = ptr_gr -> next_group;
        if (ptr_gr == nullptr) //если не найдено
        {
            wcout << L"Такой группы не найдено!" << endl;
            break;
        }
    }
    if (ptr_gr != nullptr) //если найдено - меняем номер в узле группы и у
студентов
    {
        wcout << L"Введите новый номер: ";
        cin >> ptr_gr -> gr_num;
        if (ptr_gr -> stud != nullptr)
        {

```

```

        ptr_st = ptr_gr -> stud;
        while (ptr_st != nullptr)
        {
            ptr_st -> group = ptr_gr -> gr_num;
            ptr_st = ptr_st -> next_st;
        }
    }

    wcout << L"Готово!" << endl;
}

if (choice == 9) //индивидуальное задание
{
    ind_zad(ptr_gr);
}

if (choice == 10) {break;} //выход из цикла
system("pause");
}

ptr_gr = first_group;
Node_group *temp_ptr = nullptr;
while (ptr_gr != nullptr) //удаление групп
{
    temp_ptr = ptr_gr -> next_group;
    del_gr(ptr_gr);
    ptr_gr = temp_ptr;
}

return 0;
}

```

ЗАКЛЮЧЕНИЕ

Созданная мной программа выполняет все поставленные перед ней задачи: она может создавать базу данных о студентах, позволяет редактировать и удалять информацию о них, а также выводить её в консоль и в текстовый файл. Программа соответствует всем предъявляемым требованиям.

ПРИЛОЖЕНИЕ

Примеры работы программы:

Пример организации данных в входном файле:

13

3391

Tikhonov A.A

0

2005

5

5

5

4

5

3392

Garres G.V

0

2005

4

4

4

2

4

3391

Maha D.E

3500

2004

2

3

5

2

34

Выберите действие:

1 – вывести информацию о студентах в консоль

2 – вывести информацию о студентах в файл

3 – ввести информацию о студентах с консоли

4 – ввести информацию о студентах из файла

5 – удалить информацию о группе

6 – удалить информацию о студенте

7 – редактировать информацию о студенте

8 – изменить номер группы

9 – вывести номера групп, в которых доля двоечников превышает определенный процент

10 – завершить работу программы

4|

Ввод с консоли:

```
Введите количество студентов: 1
Студент "1:
Введите номер группы: 3376
Введите ФИО студента: Nihonov N.K
Введите размер стипендии: 2750
Введите год рождения: 2004
Последовательно(через Enter) введите оценки студента за сессию:
4
5
4
5
4
Для продолжения нажмите любую клавишу . . . |
```

Вывод в консоль:

```
Группа "3391
Ermoshkin V.O
    Размер стипендии: 2900
    Возраст: 19
    Оценки за последнюю сессию: 5 4 5 4 5
Lokhmatkin K.D
    Размер стипендии: 2000
    Возраст: 20
    Оценки за последнюю сессию: 2 3 3 3 3
Prokofiev V.V
    Размер стипендии: 0
    Возраст: 19
    Оценки за последнюю сессию: 3 3 3 3 3
ghdf
    Размер стипендии: 0
    Возраст: 19
    Оценки за последнюю сессию: 5 5 5 5 5
Группа "3392
Yurieva M.E
    Размер стипендии: 0
    Возраст: 18
    Оценки за последнюю сессию: 4 4 4 2 4
Pirogova A.G
    Размер стипендии: 2700
    Возраст: 18
    Оценки за последнюю сессию: 4 4 4 4 4
Gladkikh A.R
    Размер стипендии: 0
    Возраст: 18
    Оценки за последнюю сессию: 2 3 4 4 5
Группа "3393
Kosaeva V.A
    Размер стипендии: 0
    Возраст: 18
    Оценки за последнюю сессию: 3 3 3 3 3
Konovalev A.N
    Размер стипендии: 0
    Возраст: 20
    Оценки за последнюю сессию: 3 3 3 3 3
hvhj
    Размер стипендии: 30000
    Возраст: 200
    Оценки за последнюю сессию: 3 4 3 4 3
Группа "3354
Zhavoronkova E.S
    Размер стипендии: 0
    Возраст: 18
    Оценки за последнюю сессию: 2 3 4 5 6
Для продолжения нажмите любую клавишу . . . |
```


Вывод в файл:

```
Группа №3391
Tikhonov A.A
Размер стипендии: 0
Год рождения: 2005
Оценки за последнюю сессию: 5 5 5 4 5
Maha D.E
Размер стипендии: 3500
Год рождения: 2004
Оценки за последнюю сессию: 2 3 5 2 3
Sida D.V
Размер стипендии: 2100
Год рождения: 2005
Оценки за последнюю сессию: 4 4 5 5 4
Voropaev I.N
Размер стипендии: 3200
Год рождения: 2005
Оценки за последнюю сессию: 5 5 5 5 5
Kolenko M.A
Размер стипендии: 0
Год рождения: 2000
Оценки за последнюю сессию: 2 3 4 4 4
Trapezin B.Y
Размер стипендии: 2000
Год рождения: 2005
Оценки за последнюю сессию: 5 4 4 4 4
Группа №3392
Garres G.V
Размер стипендии: 0
```

Удаление информации о группе:

```
Введите номер группы, данные о которой вы хотите удалить: 3376
Готово!
Для продолжения нажмите любую клавишу . . . |
```

Удаление информации о студенте:

```
Введите номер группы студента, информацию о котором вы хотите удалить: 3391
Введите ФИО студента: Tikhonov A.A
Готово!
Для продолжения нажмите любую клавишу . . . |
```

Редактирование информации о студенте:

```
Введите номер группы студента, информацию о которой вы хотите редактировать: 3391
Введите ФИО студента: Tikhonov A.A|
```

```
Выберите действие:  
1 – изменить размер стипендии  
2 – изменить год рождения  
3 – изменить оценки  
4 – изменить номер группы  
1  
Введите новые данные: 10000|
```

```
Выберите действие:  
1 – изменить размер стипендии  
2 – изменить возраст  
3 – изменить оценки  
4 – изменить номер группы  
4  
Введите новые данные: 3391|
```

Редактирование информации о группе(её номер):

```
Введите номер группы, номер которой хотите изменить: 3392  
Введите новый номер: 33392  
Готово!  
Для продолжения нажмите любую клавишу . . . |
```

Индивидуальное задание:

```
Введите предел доли двоечников/троечников в группе(в процентах): 34  
Группы, в которых двочеников/троечников более 34%:  
3391  
3393  
33392  
Для продолжения нажмите любую клавишу . . . |
```

```
Введите предел доли двоечников/троечников в группе(в процентах): -100  
Группы, в которых двочеников/троечников более -100%:  
3361  
3391  
3393  
33392  
Для продолжения нажмите любую клавишу . . . |
```